

樹莓派 5 Camera (rpikam-vid + SDL2) 程式碼深度解析

這份文件專為具備 **Operating System** 與 **Embedded System** 背景的工程師（如熊熊你）所撰寫。我們將從 Linux Kernel 的角度，剖析這份 User Space 程式如何運作，以及每一個 Library 與 System Call 背後的意義。

第一部分：Library 與 Linux OS 的對應關係

這段 include 區域揭示了程式所需的系統資源介面。

Library	Linux/OS 概念	深度解析
<stdio.h>	Standard I/O (libc)	提供 User Space 的緩衝區 I/O (printf , snprintf)。對應到 OS 的 File System 層，最終透過 write() syscall 輸出到 stdout。
<stdlib.h>	Memory Management	提供 Heap 記憶體操作 (malloc , free)。在 OS 層對應到 brk() 或 mmap() syscall 來向 Kernel 請求虛擬記憶體頁面 (Pages)。
<string.h>	Memory Manipulation	用於操作記憶體區塊 (strerror)。這是純 User Space 的操作，不涉及 Kernel trap。
<unistd.h>	POSIX API (System Calls)	核心關鍵。它提供了對 UNIX 標準 System Calls 的存取介面，如 fork() (行程複製), pipe() (IPC), read() , close() , dup2() (文件描述符複製), usleep() (排程讓出)。
<signal.h>	Software Interrupts	處理非同步事件 (Signals)。這是 OS 用來通知 Process 發生特定事件（如使用者按 Ctrl+C）的機制。
<sys/wait.h>	Process State	提供父行程監控子行程狀態的機制 (waitpid)。這涉及 OS 的 Process Scheduler 與 Exit Status 回收，防止產生 Zombie Process。
<errno.h>	Error Handling	存取全域錯誤碼 errno 。當 System Call 失敗時，Kernel 會設定這個值來告知 User Space 具體原因。
<fcntl.h>	File Control	控制 File Descriptor (FD) 的屬性。在此用於設定 Pipe 的 Non-blocking (O_NONBLOCK) 模式，這改變了 I/O 行為，避免 read() 卡住主執行緒。
<SDL2/SDL.h>	Graphics/HAL	Userspace Library。它封裝了底層顯示介面 (如 X11, Wayland, KMS/DRM)，負責視窗管理與像素渲染。

第二部分：程式碼邏輯與 OS 行為逐行解析

1. 全域變數與巨集定義 (Global & Macros)

```
#define WINDOW_WIDTH 640
#define WINDOW_HEIGHT 480
#define FRAME_SIZE (WINDOW_WIDTH * WINDOW_HEIGHT * 3 / 2) // YUV420
static volatile int running = 1;
```

```
static pid_t camera_pid = -1;
```

- **FRAME_SIZE 計算原理:**

- 這是 **YUV420 (I420)** 格式的定義。
- **Y (Luma):** 亮度，每個像素 1 byte ($W \times H$)。
- **U/V (Chroma):** 色度，每 2×2 個像素共用一個 U 和一個 V。所以 U 和 V 的大小各是 $W \times H/4$ 。
- **總大小:** $1 + 0.25 + 0.25 = 1.5$ (即 $\times \frac{3}{2}$)。
- **volatile 的必要性:**
 - 告訴編譯器 (GCC) **不要對這個變數進行最佳化**。
 - 因為 `running` 會在 Signal Handler (中斷環境) 被修改，如果不加 `volatile`，主迴圈可能會使用暫存器 (Register) 中的快取值，導致無法偵測到變數已被外部修改，造成程式無法停止。

2. YUV 轉 RGB 函式 (`yuv420_to_rgb`)

這是一個純 CPU 計算的函式。

- **意義:** 相機輸出的原始資料是 YUV 格式 (節省頻寬)，但 SDL Texture (預設) 與螢幕顯示通常使用 RGB 格式。
- **邏輯:** 透過色彩空間轉換公式，將 Y (亮度) 與 UV (色差) 映射回 R, G, B 三原色。
- **效能:** 這段程式碼在 Main Loop 中執行，會大量消耗 CPU 資源。在生產環境通常會使用 GPU (Shader) 或 SDL 內建的 YUV Texture 來處理，但在這裡是為了展示原理。

3. 主程式初始化 (Main Initialization)

```
// 建立管道 (Kernel Buffer)
if (pipe(pipefd) < 0) { ... }
```

- **OS 觀點:** `pipe()` 是一個 System Call。Kernel 會在核心記憶體中建立一個 **Ring Buffer (環形緩衝區)**。
- **pipefd[2]:**
 - `pipefd[0]` : 讀取端 (Read end)。
 - `pipefd[1]` : 寫入端 (Write end)。
 - 這就像在記憶體中拉了一條管子，一頭進，一頭出。

```
// 記憶體配置 (Virtual Memory)
yuv_buffer = malloc(FRAME_SIZE);
rgb_buffer = malloc(WINDOW_WIDTH * WINDOW_HEIGHT * 3);
```

- **OS 觀點:** 此時 OS 僅分配了 **虛擬地址空間 (VMA)**。真正物理記憶體 (Page Frame) 的分配會等到程式第一次寫入這些 buffer 時 (Page Fault) 才發生。

4. 關鍵核心：Fork 與 Exec (The Process Creation)

這是整份程式最精彩的地方，涉及 Linux 多工處理的核心。

```
camera_pid = fork();
```

- **fork()**: 呼叫一次，返回兩次。
- OS 會複製目前的 Process (Parent) 產生一個幾乎一模一樣的 Process (Child)。
- 此時，Parent 和 Child **共享** 打開的 File Descriptors (包含剛剛建立的 Pipe)。

子進程 (Child Process) - 生產者 (Producer)

```
if (camera_pid == 0) {
    close(pipefd[0]);           // 1. 關閉讀取端 (子進程只需要寫)
    dup2(pipefd[1], STDOUT_FILENO); // 2. 重定向 stdout
    close(pipefd[1]);           // 3. 關閉原本的寫入端 FD

    // ... 執行 rpicas-vid ...
    execlp("rpicas-vid", ...);
}
```

- `close(pipefd[0])`: 好的習慣。避免子進程意外讀取管道，並節省 File Descriptor table 資源。
- `dup2(pipefd[1], STDOUT_FILENO)`: 這是魔法所在。它將 Child Process 的 Standard Output (FD 1) **替換** 成 Pipe 的寫入端。之後任何寫入 stdout 的資料，都會直接流進 Pipe 的 Kernel Buffer。
- `execlp(...)`:
 - **OS 觀點**: 用 `rpicas-vid` 的執行檔代碼覆蓋當前 Child Process 的記憶體空間。PID 不變，但程式碼變了。
 - **參數詳解**:
 - `--codec yuv420`: 強制硬體編碼器輸出 YUV 格式。
 - `-o -`: 最重要參數。- 代表輸出到 `stdout`。
 - `-t 0`: 錄影時間無限。
 - `-n`: No preview。

父進程 (Parent Process) - 消費者 (Consumer)

```
close(pipefd[1]); // 1. 關閉寫入端
int flags = fcntl(pipefd[0], F_GETFL, 0);
fcntl(pipefd[0], F_SETFL, flags | O_NONBLOCK); // 2. 設定非阻塞
```

- `close(pipefd[1])`: **極度重要**。Pipe 的行為是：只有當所有寫入端都被關閉時，讀取端才會收到 **EOF**。Parent 不會寫入 Pipe，如果不關閉這個 FD，就算 Child 死了，Kernel 會認為「Parent 還有可能寫入」，導致 Parent 的 `read()` 永遠卡在等待。

- `O_NONBLOCK` : 預設 `read()` 是阻塞的 (Blocking)。為了讓 SDL 視窗能持續響應事件，我們不能讓程式卡在 `read()` 。

5. 主迴圈 (Event Loop & Data Pump)

```
while (running) {
    // SDL 事件處理 (UI 響應)
    while (SDL_PollEvent(&event)) { ... }

    // 檢查子進程狀態
    waitpid(camera_pid, &status, WNOHANG);

    // 讀取 Pipe
    fcntl(pipefd[0], F_SETFL, flags & ~O_NONBLOCK); // 切換回阻塞模式
    // ... 讀取迴圈 ...
}
```

- **讀取策略:** 這裡程式碼做了一個特殊的切換：`fcntl` 切回阻塞模式。
- **邏輯:** 既然確定要讀取一張完整的 Frame，為了程式簡單起見，暫時讓它 Block 住等待資料填滿 buffer。
- **Pipeline:** `read()` (Kernel to User) -> `yuv420_to_rgb` (CPU Compute) -> `SDL_UpdateTexture` (RAM to VRAM) -> `SDL_RenderPresent` (Swap Buffer)。

6. 清理與訊號處理 (Cleanup & Signals)

```
void signal_handler(int sig) { running = 0; }
void cleanup_camera(void) { kill(camera_pid, SIGTERM); ... }
```

- **Signal:** 當使用者按下 `Ctrl+C`，OS 發送 `SIGINT`。Handler 將 `running` 設為 0。
- **Graceful Shutdown:** 程式結束前，先送 `SIGTERM` (請你結束)，等待一下，若不從則送 `SIGKILL` (強制處決)。
- `waitpid`: 這是 OS 資源回收的關鍵。Child 結束後會變成 "**Zombie Process**" (殭屍行程)，直到 Parent 呼叫 `wait` 讀取它的 Exit Code，它才會真正消失。

第三部分：邏輯指南

如果你要從零開始寫這份程式，請依循以下思維順序 (Data Flow 導向)：

1. 定義資料介面 (Interface)

- 解析度 (640x480)。
- 資料量 ($YUV420 = W \times H \times 1.5$)。
- 建立 Buffer (`malloc`)。

2. 建立通訊管道 (Mechanism)

- 因為要呼叫外部程式，不能直接 Function Call。
- 建立 `pipe()`。

3. Process 分流 (Concurrency)

- 呼叫 `fork()` 。
- **Child (Worker):**
 - 把嘴巴 (`stdout`) 接到 Pipe 的入口 (`dup2`) 。
 - 變身成相機程式 (`exec1p`) 。
- **Parent (Manager):**
 - 把 Pipe 的入口關掉 (只讀不寫) 。
 - 初始化 UI (`SDL`) 。

4. 建立 Event Loop (Runtime)

- 處理 UI 事件 。
- 從 Pipe 讀取資料 (注意 TCP/Pipe 是 Stream，要確保讀滿 `FRAME_SIZE`) 。
- 資料轉換 (YUV -> RGB) 。
- 畫到螢幕上 。

5. 資源管理 (Lifecycle)

- 捕捉 `Ctrl+C` 信號 。
- 離開時記得殺死 Child Process 並回收記憶體 。

這份程式碼展示了 Linux "**Everything is a file**" 的哲學 (相機資料流就是一個 File